Taller Integrador 2 + 3

Persistencia y sincronización

Desarrollo Móvil · Santiago Cano Duque

Valor: 5%	**Modalidad:** parejas

Objetivo y contexto

Construir una aplicación Flutter pequeña para consultar un catálogo de productos y guardar una prioridad y una nota personal por producto. Integre HTTP/JSON, Drift, Repository y sincronización con Firestore.

Puede utilizar el proyecto **Movies** y las guías de clase como referencia. La solución debe aplicarse al catálogo de productos.

1. Catálogo remoto

Consuma una API de productos en formato JSON y convierta la respuesta en objetos Dart. Cada producto debe tener identificador, nombre, categoría, precio e imagen.

Muestre el catálogo y diferencie los estados de carga, error, lista vacía y datos disponibles. Controle los códigos HTTP no exitosos, errores de petición y **timeout**; permita reintentar cuando ocurra un error.

2. Detalle y validación

Al seleccionar un producto, abra su detalle y permita registrar una **prioridad de 1 a 5** y una **nota personal**. Envíe el producto seleccionado a esta pantalla.

Regla: solo se permite guardar si la prioridad es mayor o igual a 2. Si no se cumple, muestre un mensaje y no guarde la información.

3. Persistencia local con Drift

Guarde como mínimo los campos **productId, note, priority, updatedAt y pendingSync**. Cada producto debe tener un único registro vigente: al guardar nuevamente, actualice el existente sin duplicarlo.

Al volver a abrir el detalle, recupere la nota, la prioridad y el estado de sincronización desde Drift. Los datos deben conservarse al cerrar y abrir la aplicación.

4. Guardado offline-first

Guarde primero en Drift con **pendingSync = true** y después intente enviar la información a Firestore. Si la sincronización finaliza correctamente, actualice el registro local a **pendingSync = false**.

Si Firebase falla, conserve la información local y mantenga el registro pendiente. El guardado local no debe depender del éxito de Firebase.

5. Sincronización y estado

Incluya un botón **Sincronizar pendientes** que intente enviar únicamente los registros con **pendingSync == true**. Utilice el **productId** como identificador del documento en Firestore para evitar duplicados al repetir el envío.

En el detalle muestre **Sincronizado** o **Pendiente** según el valor almacenado en Drift. Si aún no existe un registro, indique que no hay información guardada.

6. Repository

Centralice en un Repository la consulta local, el guardado y la sincronización. La interfaz debe utilizar esta capa para acceder a la información personal, en lugar de coordinar directamente Drift y Firebase.

Entrega

Entregue el proyecto Flutter completo y ejecutable en un archivo **.zip**. Incluya un **README.md** con los integrantes, las instrucciones necesarias para ejecutarlo y respuestas breves, con sus propias palabras, a estas preguntas:

1. ¿Para qué sirve pendingSync?
2. ¿Por qué se guarda primero en Drift?
3. ¿Qué función cumple el Repository?
4. ¿Qué ocurre si Firebase falla?

Rúbrica

Criterio	Peso
Consumo HTTP y manejo de errores	15%
UI, navegación y validación	15%
Persistencia con Drift	25%
Repository y offline-first	20%
Firebase y sincronización	20%
Organización general	5%
Total	100%

La calificación del taller corresponde al **5% de la nota definitiva**.